

**TECNOLÓGICO DE ESTUDIOS SUPERIORES  
DEL ORIENTE DEL ESTADO DE MÉXICO**

**DESARROLLO DE UN SISTEMA INTELIGENTE DE ANÁLISIS  
DE RIESGO CREDITICIO**

MEMORIA DE RESIDENCIA PROFESIONAL

Ingeniería en Sistemas Computacionales

PRESENTA:

Villegas Rodriguez Jose Julian

Empresa: PluriOne S.A. de C.V.

Develop Talent & Technology

**BORRADOR DE TRABAJO | Septiembre de 2026**

Periodo y datos académicos pendientes de confirmación

## Nota de elaboración y datos por confirmar

Este documento es una primera versión de trabajo de la memoria de residencia profesional del proyecto de análisis de riesgo crediticio para PluriOne S.A. de C.V. Toma como referencia la organización del cuadernillo proporcionado por el alumno, correspondiente a otro autor y otro proyecto. Su extensión y evidencias crecerán con el desarrollo real del sistema.

Se conservan la presentación académica, la secuencia de diagnóstico, metodología, desarrollo y verificación, los encabezados y la numeración. El contenido se adapta al proyecto propio; no se trasladan experiencias, resultados, agradecimientos ni aprobaciones del autor del ejemplo.

### Información pendiente para la versión de entrega

| Dato                           | Situación                                                      |
|--------------------------------|----------------------------------------------------------------|
| Nombre del alumno              | Villegas Rodriguez Jose Julian; confirmar escritura y acentos. |
| Matrícula                      | Por proporcionar.                                              |
| Asesor académico y cargo       | Por proporcionar.                                              |
| Periodo y fecha de entrega     | Por confirmar; 500 horas previstas.                            |
| Aprobación institucional       | No aportada. No se incluyen firmas ni actas.                   |
| Dedicatorias y agradecimientos | Texto personal pendiente del alumno.                           |

### Criterio de redacción

Los módulos activos se describen en presente. La arquitectura futura y las actividades no realizadas se presentan como propuestas. Esta memoria no acredita entrenamiento de modelos, despliegue en Azure, capacitación, reducción del riesgo financiero ni aceptación empresarial.

La sección normativa queda pendiente de investigación y revisión institucional. Las fuentes del proyecto utilizadas aquí son el código y los documentos locales; la bibliografía técnica deberá completarse al desarrollar el marco teórico.

## Índice

| Apartado                                  | Página |
|-------------------------------------------|--------|
| Nota de elaboración y datos por confirmar | 2      |
| 1. Introducción                           | 4      |
| 2. Objetivos y justificación              | 5      |
| 3. Marco contextual                       | 6      |
| 4. Diagnóstico y requerimientos           | 7      |
| 5. Metodología y planificación            | 8      |
| 6. Tecnologías obligatorias               | 9      |
| 7. Arquitectura y organización del código | 10     |
| 8. Desarrollo e integración actual        | 11     |
| 9. Verificación y resultados disponibles  | 12     |
| 10. Mantenimiento, capacitación y cierre  | 13     |
| 11. Glosario y fuentes de información     | 14     |
| Anexo A. Orientación de la clase          | 15     |

La numeración corresponde a esta primera versión. Se ampliará conforme se incorporen desarrollo, capturas propias y evidencias de integración.

## 1. Introducción

El proyecto de Desarrollo de un Sistema Inteligente de Análisis de Riesgo Crediticio se plantea para PluriOne S.A. de C.V., cuyo nombre comercial es Develop Talent & Technology. Su propósito es construir una herramienta que apoye el análisis de información financiera y crediticia mediante una interfaz web, servicios de procesamiento de datos y, en su etapa final, modelos predictivos e integraciones con servicios especializados.

El alcance previsto comprende historiales crediticios, comportamiento de pago, ingresos, endeudamiento y variables económicas. Con esta información se busca calcular indicadores de riesgo, identificar posibles incumplimientos y presentar recomendaciones para la revisión de solicitudes y el seguimiento de créditos.

El desarrollo disponible corresponde a un prototipo local construido con React.js, Python y FastAPI. La interfaz permite capturar un identificador, ingresos, gastos y un score. El backend valida las entradas, calcula el margen mensual y devuelve una clasificación preliminar mediante reglas fijas. Este avance permite demostrar la comunicación entre cliente y servidor y la explicación de resultados.

El prototipo todavía no incorpora persistencia, autenticación, modelos entrenados ni fuentes financieras externas. Por ello, la memoria distingue el estado comprobado de la solución objetivo. Las reglas ilustrativas no se presentan como una predicción ni como una política crediticia validada por la empresa.

El documento expone el contexto empresarial, los objetivos, el diagnóstico técnico, la propuesta de gestión con Scrum, la arquitectura, la organización del código y las evidencias de verificación disponibles. Finalmente describe el trabajo pendiente de mantenimiento, capacitación y cierre, necesario para completar la entrega.

### Delimitación de esta versión

Se documenta la base funcional existente y se establece la estructura para incorporar los avances siguientes. La implementación del sistema completo requiere acuerdos sobre datos, políticas, proveedores, recursos de Azure y criterios medibles de aceptación.

## 2. Objetivos y justificación

### 2.1 Objetivo general profesional

Desarrollar un sistema inteligente para analizar el riesgo crediticio a partir de información financiera, historiales de pago y variables económicas, con resultados explicables que apoyen la revisión humana de solicitudes y su seguimiento.

### 2.2 Objetivo académico

Aplicar conocimientos de desarrollo web, ingeniería de software, bases de datos y aprendizaje automático en la construcción y documentación de una solución empresarial, organizando su avance mediante Scrum y validando sus componentes con evidencias reproducibles.

### 2.3 Objetivos específicos

1. Analizar las necesidades, las políticas de crédito y las fuentes de información disponibles, y convertirlas en requerimientos verificables.
2. Diseñar una arquitectura que articule la interfaz React, los servicios FastAPI, PostgreSQL y las integraciones obligatorias del proyecto.
3. Implementar el procesamiento de información financiera y los modelos de scoring y predicción, conservando la versión y los factores de cada evaluación.
4. Validar funcionalidad, precisión predictiva, seguridad y desempeño mediante pruebas adecuadas a cada componente.
5. Elaborar manuales técnicos y de usuario, un plan de mantenimiento y soporte, y las evidencias de capacitación y cierre.

### 2.4 Justificación

El valor esperado consiste en facilitar evaluaciones consistentes, hacer visibles los factores considerados y mejorar la trazabilidad del análisis. Para la empresa, el proyecto representa una propuesta FinTech apoyada en IA. Para el alumno, integra aprendizaje automático, analítica y arquitectura de software. Estos beneficios son objetivos por evaluar; no constituyen resultados ya demostrados.

### 3. Marco contextual

| Elemento                         | Información proporcionada                                                         |
|----------------------------------|-----------------------------------------------------------------------------------|
| Empresa / nombre comercial       | PluriOne S.A. de C.V. / Develop Talent & Technology                               |
| Domicilio                        | Puebla 46, Colonia Roma Norte, Alcaldía Cuauhtémoc, C.P. 06700, Ciudad de México. |
| RFC                              | PLU060407HC9                                                                      |
| Representante legal              | Xochicuahuatl Gleason Juárez                                                      |
| Sector y giro                    | Privado; mediana empresa. Consultoría, desarrollo de software y capacitación TI.  |
| Contacto                         | 55 1900 3503 / contacto@develop.com.mx                                            |
| Horario                          | Lunes a viernes, 09:00 a 18:00.                                                   |
| Estadías y asesoría externa      | Juan Méndez Herrera                                                               |
| Líder de proyecto / evaluaciones | Edgar Loheffelman                                                                 |

Fuente: propuesta del proyecto entregada por el alumno. Los datos empresariales se reproducen como información proporcionada, sin verificación externa.

#### 3.1 Vinculación del proyecto

La solución se relaciona con el giro de desarrollo de software y consultoría tecnológica de la empresa. El trabajo propone integrar servicios web, almacenamiento y análisis predictivo para apoyar un proceso de evaluación crediticia. No se presupone que la empresa opere actualmente una cartera de crédito ni que se haya estudiado un proceso interno que no fue documentado.

#### 3.2 Cultura organizacional y marco normativo

Se requieren misión, visión y organigrama autorizados para completar la cultura organizacional. Para el marco normativo deben identificarse las condiciones aplicables al uso de los datos y a la operación prevista, junto con su revisión correspondiente. No se declara la firma de convenios de confidencialidad ni el cumplimiento de obligaciones cuya evidencia no se ha aportado.

## 4. Diagnóstico y requerimientos

El diagnóstico parte del repositorio local. El sistema acepta cuatro entradas y produce una evaluación temporal. No se identificó almacenamiento persistente ni recuperación de expedientes. El campo denominado RFC admite texto libre no vacío; su validación no acredita identidad ni formato fiscal.

### 4.1 Proceso actual

Captura en React > envío HTTP a FastAPI > validación de entradas > cálculo de margen y reglas > respuesta JSON > presentación de riesgo y factores. Si la solicitud falla, la interfaz comunica el error. Al cambiar los datos se elimina el resultado anterior.

| Necesidad                 | Situación y criterio propuesto                                       |
|---------------------------|----------------------------------------------------------------------|
| Captura y validación      | Implementada. Rechazar valores fuera de los límites definidos.       |
| Explicación de resultados | Implementada para reglas demo; mostrar margen, factores y versión.   |
| Historial y persistencia  | Pendiente. Recuperar solicitudes y evaluaciones con fecha y versión. |
| Pagos y endeudamiento     | Pendiente. Acordar variables, fuentes y validaciones.                |
| Predicción                | Pendiente. Entrenar y contrastar con un modelo de referencia.        |
| Dashboard y alertas       | Pendiente. Métricas conciliadas y alertas trazables.                 |
| Identidad y permisos      | Pendiente. Probar roles y registrar accesos.                         |
| Integraciones financieras | Pendiente. Conservar fuente, fecha y errores de consulta.            |

### 4.2 Información necesaria

Deben acordarse la definición de incumplimiento, el horizonte de predicción, la disponibilidad de históricos, los usuarios y roles, los proveedores y los objetivos de desempeño. Sin esos acuerdos no es posible fijar umbrales productivos ni demostrar precisión del modelo.

## 5. Metodología y planificación

Scrum es un requisito del proyecto. Se propone organizar el desarrollo en iteraciones de dos semanas, ajustadas al calendario que se acuerde. El backlog prioriza incrementos verificables: base local, persistencia, acceso por roles, modelos, integraciones, reportes y entrega.

En cada iteración se deberá definir un objetivo, seleccionar trabajo, revisar el incremento y registrar mejoras. Los roles y responsables se deben acordar con la empresa. La documentación disponible plantea estas actividades, pero no acredita que ya se hayan realizado sprints o ceremonias.

### 5.1 Distribución propuesta de 500 horas

| Actividad                | Horas | Entregable                            |
|--------------------------|-------|---------------------------------------|
| Detección de necesidades | 40    | Requerimientos y fuentes              |
| Propuesta de solución    | 25    | Alcance técnico acordado              |
| Arquitectura             | 45    | Componentes, datos y contratos        |
| Planificación            | 25    | Backlog y calendario                  |
| Desarrollo               | 200   | Backend, frontend, IA e integraciones |
| Pruebas                  | 60    | Evidencias de QA                      |
| Documentación            | 35    | Manuales                              |
| Mantenimiento            | 20    | Plan de soporte                       |
| Capacitación             | 25    | Material y acta real                  |
| Cierre                   | 25    | Informe y aceptación                  |
| Total                    | 500   | Horas planificadas, no ejecutadas     |

### 5.2 Criterio de terminado

Una tarea se considerará terminada cuando su comportamiento pueda demostrarse, las verificaciones pertinentes estén aprobadas, la documentación corresponda al código y exista evidencia de revisión. Una carpeta reservada o un documento de intención no acreditan una integración funcional.

## 6. Tecnologías obligatorias

Las siguientes tecnologías forman parte del alcance requerido. La tabla diferencia lo utilizado actualmente de lo reservado para desarrollo posterior.

| Tecnología             | Función prevista                        | Estado    |
|------------------------|-----------------------------------------|-----------|
| Python                 | Reglas y procesamiento del backend      | Activo    |
| FastAPI                | Recepción y respuesta de la API         | Activo    |
| React.js               | Captura y presentación web              | Activo    |
| Azure OpenAI Service   | Explicaciones con contexto autorizado   | Pendiente |
| Azure Machine Learning | Entrenamiento, evaluación y despliegue  | Pendiente |
| Azure AI Search        | Recuperación de políticas y documentos  | Pendiente |
| PostgreSQL             | Solicitudes, historiales y evaluaciones | Pendiente |
| Power BI               | Reportes y análisis agregado            | Pendiente |
| Docker                 | Ejecución reproducible de servicios     | Pendiente |
| GitHub Actions         | Automatización de verificaciones        | Pendiente |
| Microsoft Entra ID     | Identidad y permisos de acceso          | Pendiente |
| APIs financieras       | Historiales e indicadores externos      | Pendiente |
| Scrum                  | Organización de iteraciones y revisión  | Propuesto |

### 6.1 Separación de responsabilidades

Azure Machine Learning corresponde al proceso predictivo. Azure OpenAI Service se contempla para redactar explicaciones sustentadas en los resultados y las políticas recuperadas. La explicación generada no deberá sustituir el resultado del modelo ni alterar decisiones por sí sola.

La aplicación actual ejecuta reglas locales y no invoca servicios de Azure. Los recursos, accesos y contratos necesarios deben configurarse y probarse antes de cambiar su estado a implementado.

## 7. Arquitectura y organización del código

### 7.1 Arquitectura actual y objetivo

Actualmente React consume la API FastAPI mediante HTTP. El backend separa rutas, validaciones y evaluación. Se describe como una arquitectura cliente-servidor con módulos por responsabilidad; no se atribuye una capa de persistencia que todavía no existe.

La arquitectura objetivo incorpora PostgreSQL para registros; Entra ID para identidad; Azure ML para predicción; AI Search y Azure OpenAI para consulta y explicación; APIs financieras como fuentes; y Power BI para reportes. Docker y GitHub Actions apoyarán ejecución y verificación.

| Ubicación relativa a la raíz interior        | Responsabilidad                     |
|----------------------------------------------|-------------------------------------|
| backend/app/rutas_api/                       | Endpoints HTTP                      |
| backend/app/validacion_datos/                | Esquemas de entrada                 |
| backend/app/evaluacion_creditticia/          | Reglas de demostración              |
| backend/app/persistencia_postgresql/         | Persistencia pendiente              |
| backend/app/autenticacion_entra_id/          | Identidad pendiente                 |
| backend/app/integraciones/                   | Adaptadores externos pendientes     |
| frontend/src/evaluacion_creditticia/         | Componentes y cliente de evaluación |
| frontend/src/styles/                         | Estilos de la interfaz              |
| inteligencia_artificial/prediccion_azure_ml/ | Entrenamiento pendiente             |
| analitica/reportes_power_bi/                 | Reportes pendientes                 |
| infraestructura/contenedores_docker/         | Contenedores pendientes             |
| .github/workflows/                           | Automatización pendiente            |
| gestion_proyecto/scrum/                      | Seguimiento del proyecto            |

Los directorios pendientes contienen guías de responsabilidad. No representan servicios activos. La raíz operativa está dentro de la segunda carpeta empresa - copia; los scripts locales resuelven frontend y backend a partir de ella.

## 8. Desarrollo e integración actual

### 8.1 Backend

El archivo backend/app/main.py crea la aplicación FastAPI y configura los orígenes permitidos de CORS. GET / informa disponibilidad y POST /api/v1/evaluar procesa una solicitud. La documentación del contrato está disponible en /docs cuando la API está iniciada.

El esquema SolicitudCredito exige un identificador no vacío, ingresos positivos, gastos no negativos y un score entero entre 0 y 1000. Los ingresos y gastos tienen un máximo de mil millones. Se rechazan campos adicionales y valores numéricos no finitos.

### 8.2 Evaluación ilustrativa

| Condición                                | Resultado      |
|------------------------------------------|----------------|
| Margen = ingresos menos gastos           | Margen mensual |
| Margen > 15000 y score >= 680            | Riesgo Bajo    |
| Si no cumple lo anterior y margen > 7000 | Riesgo Medio   |
| En los demás casos                       | Riesgo Alto    |

Ejemplo ficticio: ingresos de \$30,000, gastos de \$10,000 y score de 700 producen un margen de \$20,000 y riesgo Bajo. Esta salida es determinista; no equivale a una probabilidad de incumplimiento ni prueba precisión crediticia.

### 8.3 Frontend e intercambio de datos

App.jsx coordina el estado, EvaluationForm presenta la captura y EvaluationResult muestra el resumen. El cliente creditApi.js convierte los campos numéricos, envía JSON a FastAPI y gestiona errores. Durante la evaluación se deshabilita el formulario; al modificar entradas se limpia el resultado.

La aplicación debe abrirse a través del servidor Vite en <http://127.0.0.1:5173/>. Abrir index.html directamente no ejecuta el flujo de desarrollo. La API utiliza el puerto 8000. Los scripts de inicio se encuentran en scripts/.

## 9. Verificación y resultados disponibles

En la revisión previa de esta sesión, posterior a la reorganización, las cinco pruebas automatizadas del backend finalizaron correctamente; el frontend compiló y ESLint no reportó errores. Estas comprobaciones verifican aspectos del prototipo y no certifican el sistema final.

| Prueba disponible    | Comportamiento comprobado                   |
|----------------------|---------------------------------------------|
| Evaluación explicada | Margen, nivel, modo demo y factores         |
| Límites de reglas    | Umbral de margen, score y margen negativo   |
| Identificador libre  | Aceptación del formato libre documentado    |
| Entradas inválidas   | Rechazo de valores fuera de límites         |
| CORS                 | Rechazo de preflight de origen no permitido |

### 9.1 Evidencias que deben anexarse

Para la versión final se deberán adjuntar capturas propias del formulario, resultado y errores; salida de pruebas con fecha y versión del código; y evidencia de las integraciones implementadas. Las imágenes del cuadernillo de referencia pertenecen al proyecto de Evaltech y no acreditan funcionamiento de PluriOne.

### 9.2 Validaciones pendientes

Persistencia y migraciones; identidad y permisos; integración con proveedores; carga y latencia con objetivos acordados; recuperación de respaldos; aceptación de usuarios; precisión, calibración y análisis por segmentos del modelo predictivo.

### 9.3 Límites de la evidencia

La compilación demuestra que puede generarse la interfaz. Las pruebas de reglas demuestran su comportamiento definido. Ninguna de ellas prueba capacidad de predecir impagos. Esa validación requerirá datos con resultados observados y separación adecuada de entrenamiento y evaluación.

El archivo docs/QA.md conserva referencias históricas a cuatro pruebas y debe actualizarse antes de la entrega para coincidir con las cinco pruebas actuales. Este borrador registra la diferencia para evitar inconsistencias entre documentos.

## **10. Mantenimiento, capacitación y cierre**

### **10.1 Soporte propuesto**

El nivel 1 atenderá dificultades de captura y conectividad. El nivel 2 diagnosticará API, persistencia e integraciones. El nivel 3 resolverá cambios de código, modelo y seguridad, y coordinará escalamiento a proveedores. Responsables y tiempos de respuesta están pendientes de acuerdo.

El plan deberá definir respaldos, recuperación, seguimiento de errores y disponibilidad, revisión de dependencias y control de cambios. Cuando exista un modelo predictivo se deberá considerar su desempeño posterior y la actualización de datos y versiones.

### **10.2 Capacitación**

Se propone una sesión para analistas y administradores con alcance, captura, interpretación de resultados, revisión humana, acceso y reporte de incidentes. La práctica incluirá escenarios ficticios y entradas inválidas. El acta deberá registrar fecha, asistentes, versión, ejercicios y observaciones después de la sesión real.

### **10.3 Conclusiones del avance**

El proyecto dispone de una base web funcional para capturar información, validarla y mostrar una clasificación explicada. La separación del código facilita localizar las responsabilidades y continuar el desarrollo. El avance todavía no satisface el alcance predictivo, de persistencia e integración del sistema final.

La prioridad técnica siguiente es incorporar almacenamiento e historial, concretar los datos de pagos y deuda, implementar identidad y construir el proceso de entrenamiento y validación. Los reportes, servicios externos, automatización y pruebas ampliarán la evidencia de cumplimiento.

### **10.4 Condiciones de cierre**

El informe final deberá comparar objetivos con resultados medidos, incluir limitaciones, evidencias, lecciones aprendidas y aceptación empresarial. No existe evidencia aportada de entrega final o liberación, por lo que esta sección es una propuesta de cierre y no un acta de cumplimiento.

## 11. Glosario y fuentes de información

| Término            | Uso en este proyecto                                           |
|--------------------|----------------------------------------------------------------|
| API                | Interfaz HTTP para intercambiar solicitudes y resultados.      |
| Backend / frontend | Procesamiento del servidor / interfaz del usuario.             |
| Score              | Puntaje capturado; todavía no calculado por un modelo propio.  |
| Margen mensual     | Diferencia entre ingresos y gastos capturados.                 |
| Persistencia       | Conservación de registros para consulta posterior.             |
| Modelo predictivo  | Modelo aprendido de datos para estimar un resultado.           |
| Calibración        | Correspondencia entre probabilidades y frecuencias observadas. |
| Backlog            | Trabajo pendiente ordenado para planificar incrementos.        |

### 11.1 Fuentes utilizadas en este borrador

1. Villegas Carrasco Carlos Daniel (2025). Desarrollo del backend e integración de la base de datos para la plataforma web empresarial de Evaltech. Memoria de residencia profesional, TESOEM. PDF de 92 páginas proporcionado por el alumno. Utilizado como referencia de estructura y presentación.
2. Propuesta del Proyecto de Desarrollo de un Sistema Inteligente de Análisis de Riesgo Crediticio. Información empresarial, alcance y actividades proporcionados por el alumno en esta conversación.
3. Repositorio local del proyecto PluriOne: README.md, REQUISITOS\_TECNOLOGICOS.md, docs/PLAN\_PROYECTO.md, docs/OPERACION.md, docs/QA.md y docs/estadia/ALUMNO.md.
4. Código del prototipo: backend/app/, backend/tests/test\_api.py y frontend/src/. Resultados de las verificaciones realizadas en la sesión de trabajo.

### 11.2 Bibliografía por completar

El marco teórico deberá incorporar fuentes primarias verificadas de Scrum y de las tecnologías utilizadas. La sección normativa requiere investigación específica. No se trasladan las referencias sobre metodologías o telecomunicaciones del ejemplo si no sustentan el proyecto actual.

## Anexo A. Orientación de la clase

La fotografía del pizarrón proporcionada por el alumno complementa el cuadernillo de referencia. Se toma como orientación aproximada; la escritura parcialmente borrosa no permite establecer una rúbrica definitiva ni asegurar el orden o numeración de todos los apartados.

| Texto distinguible o probable       | Aplicación a la memoria de PluriOne                                                                                                         |
|-------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------|
| Conceptual / fundamentos            | Explicar los conceptos y tecnologías que sustentan el análisis de riesgo. Ampliar el marco teórico con fuentes.                             |
| Project Charter                     | Preparar la ficha de inicio: propósito, alcance, entregables, responsables, restricciones y criterios de aceptación. Pendiente de revisión. |
| WBS                                 | Desglosar el trabajo en entregables: requerimientos, arquitectura, datos, backend, frontend, IA, integraciones, QA y entrega.               |
| IEEE 830 (lectura probable)         | Confirmar con el asesor la referencia y el formato de especificación de requisitos esperado. No se declara cumplimiento de la norma.        |
| Prototipo / back / front            | Documentar la interfaz, las rutas, las reglas y la comunicación React-FastAPI con evidencias propias.                                       |
| Base de datos / diagramas (parcial) | Preparar el modelo de datos y sus relaciones para PostgreSQL. Confirmar los diagramas solicitados.                                          |
| Scrum / tablero Kanban              | Mostrar backlog y seguimiento visual de tareas con su estado real.                                                                          |
| Ceremonias / sprints                | Registrar planificación, revisión y retrospectiva cuando se realicen.                                                                       |

### A.1 Ajuste del orden de trabajo

Primero se completarán fundamentos, Project Charter y requisitos; después se documentarán arquitectura, datos y prototipo. El desarrollo se organizará con Scrum y se incorporarán pruebas y evidencias por incremento. El PDF de referencia guía la presentación, mientras que las indicaciones del asesor definen los apartados exigidos.

No se identifican con suficiente certeza todos los números, acrónimos ni notas laterales de la fotografía. Su interpretación debe confirmarse antes de cerrar el índice definitivo.